

DQN

LAMDA-5

April 3, 2026

1 DQN 算法设计

DQN (Deep Q-Network, 深度 Q 网络) 是一种将深度学习与强化学习中的 Q 学习相结合的算法。它在 2013 年由 DeepMind 提出, 并在 2015 年进行了完善, 因在 Atari 游戏上超越人类水平的表现而闻名。

在 DQN 出现之前, 传统的 Q 学习算法主要依赖于 Q table 来存储状态-动作的价值。这种方法在处理高维度、连续的状态空间 (如游戏画面、机器人视觉输入) 时会遇到根本性困难:

- 维度灾难: 如果直接用表格存储, 状态空间会巨大无比 (例如, 一个 Atari 游戏的画面像素组合几乎是无限的), 导致 Q 表无法构建和更新。
- 泛化能力差: 表格方法无法将学到的知识从见过的状态推广到未见过的相似状态。

为了解决上述问题, DQN 创新性地引入了 2 个重要机制:

- replay buffer: 强化学习采集的样本 (状态、动作、奖励、下一个状态) 通常是序列相关的, 如果直接按顺序学习, 会导致模型训练不稳定。DQN 引入 replay buffer, 将智能体与环境互动的样本存储起来, 训练时从中随机采样。这打破了数据之间的相关性, 使得数据更接近独立同分布, 提高了学习的稳定性。
- target network: 在 Q-learning 中, 如果用一个不断更新的网络来计算当前的 Q 值和目标 Q 值, 两者会相互影响, 导致 Moving Target 的情况, 容易引发学习效果发散。由此 DQN 使用两个结构相同但参数更新频率不同的网络——评估网络和目标网络。目标网络的参数在一段时间内保持不变, 每隔固定步数才从评估网络复制过来。这暂时固定了优化目标, 显著提升了算法的收敛稳定性。

2 Playing Atari with Deep Reinforcement Learning

仔细阅读文章 “Playing Atari with Deep Reinforcement Learning”, 重点关注实验部分 (部分网络架构和参数已在文章中给出)。

3 TODO

具体实现部分参考代码中的 README, 主要步骤为配置环境、完成代码中 TODO 部分, 实现并回答下述问题:

1. 在文件 `/utils/networks` 中实现 CNN 加全连接层网络初始化, 以及前向传播。
这部分的架构在 Playing Atari with Deep Reinforcement Learning 中已给出, 找到并复现。
2. 在文件 `/agent/atari_agent` 中实现 Q 值的计算以及动作的选择, 最后得到 Q 值查询接口。
 - `__init__`: 初始化参数。
 - `select_action`: 利用上面实现的 CNN 加全连接层网络我们可以从数据集 (图像) 中获得 Q 函数的计算, 并分别实现确定性的动作选择

$$\pi_{deterministic}(a | s) = \arg \max_{a'} Q(s, a')$$

或是 softmax 概率探索策略

$$\pi_{softmax}(a | s) = \frac{\exp(Q(s, a)/\alpha)}{\sum_{a'} \exp(Q(s, a')/\alpha)}$$

softmax 概率探索策略能增加探索性，但是在使用 ϵ -greedy 的情况下一般不需要再使用 softmax 探索

- `get_q, get_max_q`: 实现 Q 值查询接口，分别计算 $Q(s, a)$ 和 $\max_{a'} Q(s, a')$

3. 在文件 `/algorithm/offpolicy_algorithm/dqn` 中实现 epsilon-greedy 策略选择随机动作, 并通过均方误差和梯度反向传播更新 Q 网络。

- `__init__`: 读取 DQN 超参数，定义目标网络的更新方式（软/硬更新）并根据配置选择优化器。
- `_target_hard_update`: 每隔固定的步数直接将当前网络的参数复制给目标网络。
- `_target_soft_update`: 在每个训练步中，用当前网络参数缓慢更新目标网络参数。

$$\theta_{\text{target}} \leftarrow \tau \theta + (1 - \tau) \theta_{\text{target}}$$

其中 τ 是一个很小的常数（一般取 $\tau = 0.001$ 或 $\tau = 0.005$ ）

- `_update_policy`: 这个函数是 DQN 算法的主体，负责从回放缓冲区采样、计算损失并更新网络。

其中单个 (s, a) 的目标值 `target` 计算为：

这里的目标值计算需要使用 `target network`，并且不需要反向传播梯度；全部误差需要从 `replay buffer` 采样并计算均方误差：

$$L = \mathbb{E}_{(s, a, r, s') \sim \mathcal{D}} [(y - Q(s, a))^2]$$

- `random_choose_action`: 这一部分我们需要实现 ϵ -greedy 策略，同时引入线性退火：

$$\epsilon = \max \left(\epsilon_{\text{end}}, \epsilon_{\text{start}} - \frac{t}{T} (\epsilon_{\text{start}} - \epsilon_{\text{end}}) \right)$$

4 实验与反思

4.1 训练结果

默认训练环境是 Pong，实现后可以先 Debug 过一遍代码流程，确保代码能跑通，确保训练过程中 `loss`、`q_mean` 和 `reward` 等训练曲线合理，同时测试性能在上升，并且最终能到 DQN 文章中的测试分数 (Pong 为 21 分)。

4.2 调参

调试参数，尝试修改各种参数例如学习率、EPS 等，也可以尝试 n-step 实现，请至少尝试 2 个方向测试参数对算法性能的影响并尝试分析原因。其中 n-step 的数学公式如下：

$$G_{t:t+n} = r_{t+1} + \gamma r_{t+2} + \dots + \gamma^{n-1} r_{t+n} + \gamma^n V(S_{t+n})$$

4.3 实际环境

在其他环境上测试 DQN 效果，可以自己任选一到两个环境（例如 Breakout、SpaceInvaders、Seaquest 等）去做测试，让实验效果和文章中的性能接近。